Program Engineering Assignment 4

Part A: Test Scenarios

Scenario Name: Customer booking an appointment.

Preconditions:

1. The app is installed and running.
2. The customer is logged in.
3. At least one active branch exists in Firestore.
4. At least one active barber is linked to at least one branch.
5. There are barbers registered to said branches.

Steps:

1. A customer logs in the app.
2. Customer opens Booking screen.
3. Customer taps Choose branch (or Book appointment, depending on your UI)
4. They are directed to the map which allows them to see the branches in the database.
5. They select a branch.
6. They are directed back to the booking screen with their selected branch.
7. They select a barber from the list of barbers in the branch.
8. They choose a date and a time for the appointment.

Expected Result:

1. A new document is created in appointments with:
   userId, userEmail, branchId, branchName, barberId, barberName, date, time, and status = "PENDING".
2. The booking screen confirms success (toast/snackbar).
3. The appointment becomes visible in the barber's appointments list.

Scenario Name: Barber schedules and completes an appointment
Preconditions:

1. The barber is logged in.
2. There is at least one appointment in appointments where barberId == barberUID and status == "PENDING".

Steps:

1. Barber opens Appointments screen.

2. Barber finds a PENDING appointment in the list.
3. Barber taps Approve.
4. The appointment status updates to APPOINTMENT_SCHEDULED.
5. After the haircut is completed, barber taps Done.

Expected Result:

- On Approve: the appointment document updates status = "APPOINTMENT_SCHEDULED" (and updatedAt if you use it).
- On Done: the appointment document updates status = "DONE".
- The appointment disappears from the barber's active list (if your UI hides DONE).
- A rating request message is created for the customer under:
- users/{userId}/messages/{messageId} with type = "RATE_REQUEST" and the appointment details.

Scenario Name: Customer rates a barber after appointment completion
Preconditions:

1. Customer is logged in.
2. There is a message under users/{customerUid}/messages with type = "RATE_REQUEST" and a valid appointmentId + barberId.
3. The appointment referenced exists and has status == "DONE".

Steps:

1. Customer opens Messages screen.
2. Customer sees a rating request message ("Please rate...").
3. Customer taps Rate now.
4. A rating dialog appears with 0–5 stars (or 1–5, depending on your rules).
5. Customer selects a rating and taps Submit.

Expected Result:

- The appointment document updates with:
    - rating = selectedValue
    - ratedAt = timestamp
- The barber's rating aggregates update (either directly or via Cloud Function):
    - ratingSum += rating

    - ratingCount += 1
- The rating request message is deleted from the customer's messages list.
- The customer sees a confirmation.

Part B: Unit Tests

```
package com.example.barber4u.data.repositories;

import static org.junit.Assert.assertEquals;
import static org.mockito.Mockito.*;

import com.google.firebase.firestore.DocumentSnapshot;

import org.junit.Before;
import org.junit.Test;

public class ParseRoleTest {

  private UserRepository repo;

  @Before
  public void setUp() {
    // 1) Setup conditions
    repo = new UserRepository();
  }

  @Test
  public void parseRole_normalizesAndDefaultsCorrectly() {
    // 1) Setup conditions (mock DocumentSnapshot return values)
    DocumentSnapshot docAdmin = mock(DocumentSnapshot.class);
```

```
        when(docAdmin.getString("role")).thenReturn("  admin  ");

        DocumentSnapshot docBarber = mock(DocumentSnapshot.class);
        when(docBarber.getString("role")).thenReturn("barber");

        DocumentSnapshot docMissing = mock(DocumentSnapshot.class);
        when(docMissing.getString("role")).thenReturn(null);

        DocumentSnapshot docInvalid = mock(DocumentSnapshot.class);
        when(docInvalid.getString("role")).thenReturn("manager");

        // 2) Call function under test
        String r1 = repo.parseRole(docAdmin);
        String r2 = repo.parseRole(docBarber);
        String r3 = repo.parseRole(docMissing);
        String r4 = repo.parseRole(docInvalid);

        // 3) Assertions
        assertEquals(UserRepository.ROLE_ADMIN, r1);
        assertEquals(UserRepository.ROLE_BARBER, r2);
        assertEquals(UserRepository.ROLE_CUSTOMER, r3);
        assertEquals(UserRepository.ROLE_CUSTOMER, r4);
    }
}

package com.example.barber4u.adapters;

import static org.junit.Assert.assertEquals;
```

```
import static org.mockito.Mockito.*;

import com.example.barber4u.models.Appointment;

import org.junit.Before;
import org.junit.Test;

import java.lang.reflect.Constructor;
import java.lang.reflect.Field;
import java.util.List;

public class removeByIdTest {

  private BarberAppointmentsAdapter adapter;

  @Before
  public void setUp() throws Exception {
    // 1) Setup conditions
    adapter = spy(constructAdapterWithMocks());

    // IMPORTANT: Prevent RecyclerView internals from running in unit test
    // (this avoids the mObservers null crash)
    doNothing().when(adapter).notifyItemRemoved(anyInt());

    // Seed adapter internal items list
    List<Appointment> items = getItemsList(adapter);

    Appointment a1 = mock(Appointment.class);
```

```
        when(a1.getId()).thenReturn("1");

        Appointment a2 = mock(Appointment.class);
        when(a2.getId()).thenReturn("2");

        items.add(a1);
        items.add(a2);

        assertEquals(2, items.size());
    }

    @Test
    public void removeById_removesCorrectItem_andCallsNotifyItemRemoved()
throws Exception {
        // 2) Call function under test
        adapter.removeById("1");

        // 3) Assertions
        List<Appointment> items = getItemsList(adapter);
        assertEquals(1, items.size());

        verify(adapter).notifyItemRemoved(0);
    }

    // ---- Helpers ----

    private static BarberAppointmentsAdapter constructAdapterWithMocks()
throws Exception {
```

```
    Constructor<?>[] ctors =
BarberAppointmentsAdapter.class.getDeclaredConstructors();
    if (ctors.length == 0) throw new IllegalStateException("No constructor
found");

    Constructor<?> ctor = ctors[0];
    ctor.setAccessible(true);

    Class<?>[] paramTypes = ctor.getParameterTypes();
    Object[] args = new Object[paramTypes.length];

    for (int i = 0; i < paramTypes.length; i++) {
      Class<?> t = paramTypes[i];
      if (t.isPrimitive()) args[i] = defaultPrimitiveValue(t);
      else args[i] = mock(t);
    }

    return (BarberAppointmentsAdapter) ctor.newInstance(args);
  }

  @SuppressWarnings("unchecked")
  private static List<Appointment> getItemsList(BarberAppointmentsAdapter
adapter) throws Exception {
    Field f = BarberAppointmentsAdapter.class.getDeclaredField("items");
    f.setAccessible(true);
    return (List<Appointment>) f.get(adapter);
  }

  private static Object defaultPrimitiveValue(Class<?> t) {
```

```
        if (t == boolean.class) return false;
        if (t == byte.class) return (byte) 0;
        if (t == short.class) return (short) 0;
        if (t == int.class) return 0;
        if (t == long.class) return 0L;
        if (t == float.class) return 0f;
        if (t == double.class) return 0d;
        if (t == char.class) return '\0';
        return 0;
    }
}

package com.example.barber4u.common;

import static org.junit.Assert.*;
import static org.mockito.ArgumentMatchers.*;
import static org.mockito.Mockito.*;

import com.google.android.gms.tasks.OnFailureListener;
import com.google.android.gms.tasks.OnSuccessListener;
import com.google.android.gms.tasks.Task;
import com.google.firebase.Timestamp;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.firestore.CollectionReference;
import com.google.firebase.firestore.DocumentReference;
import com.google.firebase.firestore.FieldValue;
import com.google.firebase.firestore.FirebaseFirestore;
import com.google.firebase.firestore.WriteBatch;
```

```
import org.junit.Before;
import org.junit.Test;
import org.mockito.ArgumentCaptor;
import org.mockito.MockedStatic;

import java.lang.reflect.Field;
import java.lang.reflect.Method;
import java.util.Map;

public class SubmitRatingTest {

  private MessagesFragment fragment;        // spy
  private FirebaseFirestore db;             // mock
  private WriteBatch batch;                 // mock

  // Firestore chain mocks
  private CollectionReference apptsCol;
  private DocumentReference apptDoc;

  private CollectionReference usersCol;
  private DocumentReference userDoc;
  private CollectionReference messagesCol;
  private DocumentReference msgDoc;

  @Before
  public void setUp() throws Exception {
    // ---- 1) Setup conditions ----
```

```
fragment = spy(new MessagesFragment());

// Avoid Toast/requireContext inside success/failure callbacks in a JVM unit test
// (submitRating checks isAdded() before showing Toast)
doReturn(false).when(fragment).isAdded();

db = mock(FirebaseFirestore.class);
batch = mock(WriteBatch.class);

apptsCol = mock(CollectionReference.class);
apptDoc = mock(DocumentReference.class);

usersCol = mock(CollectionReference.class);
userDoc = mock(DocumentReference.class);
messagesCol = mock(CollectionReference.class);
msgDoc = mock(DocumentReference.class);

// Inject mocked db into fragment (db is a private field)
Field dbField = MessagesFragment.class.getDeclaredField("db");
dbField.setAccessible(true);
dbField.set(fragment, db);

when(db.batch()).thenReturn(batch);

// appointments/{appointmentId}
when(db.collection("appointments")).thenReturn(apptsCol);
when(apptsCol.document(anyString())).thenReturn(apptDoc);
```

```
    // users/{uid}/messages/{msgId}
    when(db.collection("users")).thenReturn(usersCol);
    when(usersCol.document(anyString())).thenReturn(userDoc);
    when(userDoc.collection("messages")).thenReturn(messagesCol);
    when(messagesCol.document(anyString())).thenReturn(msgDoc);

    // batch.commit() returns Task<Void>, and submitRating chains listeners on
it
    @SuppressWarnings("unchecked")
    Task<Void> task = mock(Task.class);
    when(batch.commit()).thenReturn(task);
    when(task.addOnSuccessListener(any(OnSuccessListener.class))).thenRetu
rn(task);
    when(task.addOnFailureListener(any(OnFailureListener.class))).thenReturn(t
ask);
  }

  @Test
  public void
submitRating_updatesAppointment_deletesMessage_andCommitsBatch()
throws Exception {
    // ---- 1) Setup conditions ----
    MessageItem msg = new MessageItem(
        "msg123",
        "text",
        "appt999",
        "barber42",
        "Barber Name"
```

```
    );
    int rating = 4;

    // FirebaseAuth.getInstance().getUid() is called inside submitRating.
    // We mock it to return a known uid.
    FirebaseAuth authMock = mock(FirebaseAuth.class);
    when(authMock.getUid()).thenReturn("user123");

    try (MockedStatic<FirebaseAuth> mocked = mockStatic(FirebaseAuth.class))
{
      mocked.when(FirebaseAuth::getInstance).thenReturn(authMock);

      // ---- 2) Call function under test (private method via reflection) ----
      Method m = MessagesFragment.class.getDeclaredMethod(
            "submitRating",
            MessageItem.class,
            int.class
      );
      m.setAccessible(true);
      m.invoke(fragment, msg, rating);
    }

    // ---- 3) Assertions ----

    // verify correct appointment doc used
    verify(apptsCol).document("appt999");
    verify(batch).update(eq(apptDoc), anyMap());
```

```
    // capture the map passed to batch.update and assert rating key/value exists
    @SuppressWarnings("unchecked")
    ArgumentCaptor<Map<String, Object>> mapCaptor =
ArgumentCaptor.forClass(Map.class);
    verify(batch).update(eq(apptDoc), mapCaptor.capture());
    Map<String, Object> updateMap = mapCaptor.getValue();

    assertEquals("rating must be set", rating, updateMap.get("rating"));
    assertTrue("ratedAt must exist", updateMap.containsKey("ratedAt"));
    assertNotNull("ratedAt value must not be null", updateMap.get("ratedAt"));
    // (We don't compare exact FieldValue object; just ensure it is present.)

    // verify correct message doc deleted
    verify(usersCol).document("user123");
    verify(messagesCol).document("msg123");
    verify(batch).delete(msgDoc);

    // verify commit called and listeners attached
    verify(batch).commit();
  }
}
```

Part C: UI Tests

```
package com.example.barber4u.customer;

import static androidx.test.espresso.Espresso.onView;
import static androidx.test.espresso.action.ViewActions.click;
import static androidx.test.espresso.assertion.ViewAssertions.matches;
import static androidx.test.espresso.matcher.ViewMatchers.isNotEnabled;
import static androidx.test.espresso.matcher.ViewMatchers.withId;
import static androidx.test.espresso.matcher.ViewMatchers.withText;
import static org.hamcrest.Matchers.not;
import static androidx.test.espresso.matcher.ViewMatchers.withClassName;

import androidx.fragment.app.testing.FragmentScenario;
import androidx.test.ext.junit.runners.AndroidJUnit4;

import com.example.barber4u.R;

import org.junit.Before;
import org.junit.Test;
import org.junit.runner.RunWith;

import androidx.test.espresso.contrib.PickerActions;

@RunWith(AndroidJUnit4.class)
public class BookFragmentUiTest {

  @Before
  public void launchFragment() {
```

```
    // Launch fragment in isolation (no Activity needed)
    FragmentScenario.launchInContainer(BookFragment.class);
  }

  @Test
  public void dateAndTimeSelection_updatesFields() {
    // Click date field → pick a date → confirm
    onView(withId(R.id.etDate)).perform(click());
    onView(withClassName(org.hamcrest.Matchers.equalTo("android.widget.Da
tePicker")))
        .perform(PickerActions.setDate(2026, 2, 1));
    onView(withText("OK")).perform(click());

    // Click time field → pick a time → confirm
    onView(withId(R.id.etTime)).perform(click());
    onView(withClassName(org.hamcrest.Matchers.equalTo("android.widget.Ti
mePicker")))
        .perform(PickerActions.setTime(10, 30));
    onView(withText("OK")).perform(click());

    // Verify fields are not empty
    onView(withId(R.id.etDate)).check(matches(not(withText(""))));
    onView(withId(R.id.etTime)).check(matches(not(withText(""))));
  }

  @Test
  public void bookButton_remainsDisabled_withoutBranchAndBarber() {
    // Initially disabled
    onView(withId(R.id.btnBook)).check(matches(isNotEnabled()));
```

```
    // Select date
    onView(withId(R.id.etDate)).perform(click());
    onView(withClassName(org.hamcrest.Matchers.equalTo("android.widget.DatePicker")))
        .perform(PickerActions.setDate(2026, 2, 1));
    onView(withText("OK")).perform(click());

    // Select time
    onView(withId(R.id.etTime)).perform(click());
    onView(withClassName(org.hamcrest.Matchers.equalTo("android.widget.TimePicker")))
        .perform(PickerActions.setTime(10, 30));
    onView(withText("OK")).perform(click());

    // Still disabled because branch + barber not chosen
    onView(withId(R.id.btnBook)).check(matches(isNotEnabled()));
  }
}

package com.example.barber4u.customer;

import static androidx.test.espresso.Espresso.onView;
import static androidx.test.espresso.action.ViewActions.click;
import static androidx.test.espresso.assertion.ViewAssertions.matches;
import static androidx.test.espresso.matcher.ViewMatchers.isEnabled;
import static androidx.test.espresso.matcher.ViewMatchers.isNotEnabled;
import static androidx.test.espresso.matcher.ViewMatchers.withId;
import static androidx.test.espresso.matcher.ViewMatchers.withText;
```

```
import android.os.IBinder;
import android.view.WindowManager;

import androidx.fragment.app.testing.FragmentScenario;
import androidx.test.ext.junit.runners.AndroidJUnit4;

import com.example.barber4u.R;

import org.hamcrest.Description;
import org.hamcrest.TypeSafeMatcher;
import org.junit.Before;
import org.junit.Test;
import org.junit.runner.RunWith;

@RunWith(AndroidJUnit4.class)
public class BranchSelectionFragmentUiTest {

  @Before
  public void launchFragment() {
    FragmentScenario.launchInContainer(BranchSelectionFragment.class);
  }

  @Test
  public void selectButton_isDisabledInitially() {
    // btnSelectBranch is setEnabled(false) in onViewCreated
    onView(withId(R.id.btnSelectBranch)).check(matches(isNotEnabled()));
  }
```

```
  @Test
  public void clickingSelect_withoutChoosingBranch_showsToast() {
    // We want to click the button, but it is disabled by default.
    // So this test validates the "Choose a branch" behavior by enabling button
    // (UI tests can do controlled setup via scenario.onFragment).
    FragmentScenario<BranchSelectionFragment> scenario =
        FragmentScenario.launchInContainer(BranchSelectionFragment.class);

    scenario.onFragment(fragment -> {
      // enable button so we can click it; keep selectedBranch == null
      fragment.requireView().findViewById(R.id.btnSelectBranch).setEnabled(tru
e);
    });

    onView(withId(R.id.btnSelectBranch)).perform(click());

    // Toast text from returnSelectedBranch(): "Choose a branch"
    onView(withText("Choose a branch"))
        .inRoot(new ToastMatcher())
        .check(matches(withText("Choose a branch")));
  }

  // -------- Toast matcher (standard Espresso pattern) --------
  public static class ToastMatcher extends
TypeSafeMatcher<androidx.test.espresso.Root> {
    @Override
    public void describeTo(Description description) {
      description.appendText("is toast");
```

```
    }

    @Override
    public boolean matchesSafely(androidx.test.espresso.Root root) {
      int type = root.getWindowLayoutParams().get().type;
      if ((type == WindowManager.LayoutParams.TYPE_TOAST)
          || (type ==
WindowManager.LayoutParams.TYPE_APPLICATION_OVERLAY)
          || (type ==
WindowManager.LayoutParams.TYPE_APPLICATION_PANEL)) {
        IBinder windowToken = root.getDecorView().getWindowToken();
        IBinder appToken = root.getDecorView().getApplicationWindowToken();
        return windowToken == appToken;
      }
      return false;
    }
  }
}

package com.example.barber4u.common;

import static androidx.test.espresso.Espresso.onView;
import static androidx.test.espresso.action.ViewActions.click;
import static androidx.test.espresso.assertion.ViewAssertions.doesNotExist;
import static androidx.test.espresso.assertion.ViewAssertions.matches;
import static androidx.test.espresso.matcher.RootMatchers.isDialog;
import static androidx.test.espresso.matcher.ViewMatchers.isDisplayed;
import static androidx.test.espresso.matcher.ViewMatchers.withText;
```

```
import androidx.fragment.app.testing.FragmentScenario;
import androidx.test.ext.junit.runners.AndroidJUnit4;

import org.junit.Test;
import org.junit.runner.RunWith;

@RunWith(AndroidJUnit4.class)
public class MessagesFragmentUiTest {

  @Test
  public void showRatingDialog_opensDialog_andCancelCloses() {

    // Launch the fragment in the default test container
    FragmentScenario<MessagesFragment> scenario =
        FragmentScenario.launchInContainer(MessagesFragment.class);

    // Trigger the dialog from the fragment
    scenario.onFragment(fragment -> {
      MessageItem msg = new MessageItem(
          "msg1",
          "hello",
          "appt1",
          "barber1",
          "David"
      );
      fragment.onRateNow(msg);
    });
```

```
        // Verify dialog appears
        onView(withText("Rate David"))
            .inRoot(isDialog())
            .check(matches(isDisplayed()));

        // Click Cancel
        onView(withText("Cancel"))
            .inRoot(isDialog())
            .perform(click());

        // Verify dialog is dismissed
        onView(withText("Rate David"))
            .inRoot(isDialog())
            .check(doesNotExist());
    }
}
```

Part D: Code Reviews

Review 1:

Code supplied by Emuna Sofer, ID: 213204837.

Reviewed by Eitan Beriy, ID: 326244134.

Module Reviewed: KitchenFragment.java.

Code Readability

The code is generally readable and makes good use of descriptive method names such as showNotificationsDialog() and showAddIngredientsDialog(). However, variable naming is inconsistent. For example, stock_list uses snake_case while most other variables follow camelCase. Java and Android conventions recommend camelCase throughout, so stockList would be more appropriate.

Some variable names do not accurately reflect their role. For instance, tvAmount represents user input but is named as if it were a static text field. Renaming it to etAmount would better convey its purpose. Additionally, generic parameter names such as consumer reduce clarity and could be replaced with more descriptive names like onSubmitItem.

Documentation is minimal. Complex methods such as ingredientDialog() and showAddIngredientsDialog() would benefit from brief comments or Javadoc explaining their purpose, inputs, and assumptions.

Improving Structure (DRY and Modularity)

Several methods in the fragment are large and handle multiple responsibilities. The method ingredientDialog() performs UI inflation, spinner setup, create/edit mode logic, input validation, and object creation. This violates the Single Responsibility Principle.

The structure could be improved by splitting this method into smaller helper methods, such as:

- a method for configuring spinners,
- a method for binding create-mode UI,
- a method for binding edit-mode UI,
- and a method for validating and parsing user input.

This would reduce code duplication, improve readability, and make the logic easier to test and maintain. Extracting dialog-related logic into a separate helper class (for example, KitchenDialogs) would further improve modularity and keep the fragment focused on orchestration rather than UI construction.

Bug Detection and Efficiency

Input validation in dialogs is incomplete. While numeric validation exists for ingredient amounts, there is no validation for empty ingredient names or non-positive values. This can lead to invalid data being stored or displayed.

From a performance perspective, showAddIngredientsDialog() uses a list to check whether ingredient keys already exist. This results in repeated linear searches and degrades performance as data size grows. Using a Set for membership checks would improve efficiency.

There is also a risk of redundant observers being registered for notifications. Observing the same LiveData both in onViewCreated() and again inside showNotificationsDialog() may lead to duplicated updates and unintended behavior.

Coding Standards and Conventions

The code mixes usage of var with explicit type declarations. For consistency and clarity it is recommended to use explicit types throughout.

Access modifiers are generally applied correctly, but grouping related helper methods together and documenting public static methods would improve organization. Several methods exceed a reasonable length and should be refactored to enhance maintainability.

3 Key Comments

1. Improve naming consistency and conventions
   Variable naming should consistently follow camelCase, and names should accurately describe their purpose. This enhances readability and aligns with Java and Android standards.
2. Refactor large methods to improve modularity
   Methods handling multiple responsibilities should be split into smaller, focused units. This improves maintainability, testability, and adherence to DRY principles.
3. Fix validation gaps and optimize data handling
   Additional validation is needed for user input, and inefficient list-based membership checks should be replaced with set-based logic to improve performance and reliability.

Example of Improvement

One concrete improvement involved optimizing ingredient filtering in showAddIngredientsDialog(). The original implementation stored existing ingredient keys in a list and used repeated contains() checks when filtering new ingredients. This resulted in unnecessary quadratic time complexity.

The logic was improved by storing existing keys in a Set, allowing constant-time membership checks. This change improved performance, clarified intent, and made the code more scalable as the number of ingredients increases.

Review 2:

Code supplied by Nooran Darusha, ID: 213346877.

Reviewed by Mohamad Mousa, ID: 211711072.

Module Reviewed: ProviderMyProfileScreen.kt.

Code Readability

The file is readable overall and the UI is clearly separated into view mode and edit mode, which helps comprehension. Variable names like selectedSpecializations and availableWorkAreas are descriptive.

Some improvements are recommended:

- Several strings are hardcoded (e.g., "My Profile", "Save changes"). In Android projects, these are usually moved to strings.xml for localization and consistency.
- Lists like availableSpecializations and availableWorkAreas are embedded in the composable. This makes the composable long and harder to scan. Moving them to a constant object (or repository/config) would improve clarity.
- The function is quite large and contains UI, state, and data-loading logic in one place. Splitting into smaller composables (e.g., ProfileViewCard(...), ProfileEditForm(...)) would improve readability.

Improving Structure (DRY and Modularity)

There is repeated selection logic in the FilterChip blocks:

if (selected.contains(x)) selected.remove(x) else selected.add(x).

This duplication can be removed by extracting a small helper function (or extension) that toggles membership.

The composable handles:

- loading Firestore data,
- holding form state,
- rendering UI,
- saving profile.

This is workable for a small screen, but structurally it aligns more with “all-in-one UI” than MVVM. A cleaner structure would be:

- state + loading/saving handled by a ViewModel,
- composable purely renders UI and calls events.

This would also simplify testing.

Bug Detection and Efficiency

Several real issues exist:

1. **No error handling for load phase**
    - If getUserById(uid) throws or returns null due to error, the UI silently stops loading and shows empty data.
    - A UI-visible error state (snackbar/text) should exist.
2. **Saving without loading state / double-click protection**
    - The save button can be clicked multiple times quickly.
    - No isSaving state is used, so duplicate writes can occur.
3. **Experience input is not validated**
    - experience.toIntOrNull() ?: 0 silently converts invalid input to 0.
    - This can overwrite real values unintentionally.
    - Input should restrict to digits or show an error if invalid.
4. **State reset behavior**
    - Cancel just sets isEditing = false but does **not** restore the original values from the database.
    - If the user edits fields then presses Cancel, the composable keeps the edited values in memory, and view mode will show them even though they weren't saved.
    - A snapshot of "original profile" should be stored and restored on cancel.
5. **Large lists created inside composable**
    - Lists are recreated each recomposition (even if stable). This isn't catastrophic, but moving them out of the composable or wrapping them in remember { ... } improves stability and avoids repeated allocations.

Coding Standards and Conventions

- The composable is long (high complexity). Extracting sections into smaller composables improves maintainability.
- A more idiomatic approach is to represent the profile as a data class state object rather than many independent mutableStateOf.
- On error, the code calls printStackTrace() which isn't user-friendly. A UI message should be shown.

3 Key Comments

1. **Cancel does not revert edits**
    - The screen should restore the original loaded profile data when Cancel is pressed, otherwise it misleads the user.
2. **Add loading/error/saving states**
    - Loading should display errors, saving should disable inputs and show progress to prevent duplicate writes and silent failures.

3. **Refactor into smaller composables + reduce repeated chip toggle logic**
    - Extract view and edit parts into separate composables, and deduplicate chip toggle code for better modularity.

Example of Improvement

Instead of only doing: isEditing = false.

A copy of the originally loaded profile values would be stored (e.g., originalProfileState), and Cancel would reset all fields back to those original values before leaving edit mode. This prevents unsaved data from appearing in view mode and improves correctness.

Code Review – ChatRepository.kt

יובל פרנקל-209769264 לאחמד בשיר - 213552698

Module Overview

The ChatRepository class is responsible for managing all chat-related data operations in the application using Firebase Firestore. It handles creating or retrieving chat sessions, sending messages, listening to real-time message updates, and retrieving the list of chats for a specific user. This module contains meaningful business logic, interacts with external services (Firestore), and manages complex data objects such as Chat and Message, which makes it a suitable and important module for a code review.

Code Readability

Overall, the code is readable and well-organized. Function names such as createOrGetChat, sendMessage, and listenToMessages clearly describe their purpose, which makes the code easy to understand. Variable names are meaningful and consistent, and the flow of each function is straightforward. However, the code lacks documentation comments , which could help future developers understand assumptions such as the Firestore structure or the fact

that requestId is used directly as the chat ID.

Improving Structure and Modularity

The structure of the class is generally good, with each function handling a single responsibility. However, there is some repetition in mapping Firestore documents to domain objects in both listenToMessages and listenToUserChats. Extracting this mapping logic into a small helper function would improve modularity and reduce duplication. Additionally, adding brief comments explaining the Firestore collections and fields would further improve maintainability.

Bug Detection and Efficiency

One potential issue exists in the createOrGetChat function, where a chat document is first read and then created if it does not exist. In a concurrent scenario, two users could attempt to create the same chat simultaneously, which may lead to race conditions. Using a Firestore transaction or a create-only operation would make this logic safer.

Coding Standards and Best Practices

The code follows Kotlin and Android best practices, including coroutine based asynchronous calls and clean separation of data logic into a repository layer, which aligns well with the MVVM architecture. Error handling through callback functions is a good practice, but consistency could be improved by handling all failure cases explicitly. Overall, the code adheres to standard conventions and demonstrates a good understanding of Firestore usage patterns.

Three Key Review Comments

First, the chat creation logic should be made concurrency safe to prevent race

conditions when multiple users attempt to create the same chat at the same time.

Second, the sendMessage function should provide feedback when a message is not sent for example, when the message is empty or the chat does not exist, instead of failing silently. Third, query efficiency can be improved by moving sorting logic such as ordering chats by lastMessageAt to the Firestore query instead of sorting on the client side.

Example of Improvement Based on Review

An example improvement based on this review would be modifying the createOrGetChat method to use a Firestore transaction. By performing the existence check and creation inside a transaction, the code ensures that only one chat document is created even if multiple clients attempt to create it at the same time. This change improves data consistency and makes the chat creation logic more reliable in real-world concurrent usage scenarios.